Production Agent Assessment — Sample

Subject: `agent-control-lab` reference system **Date:** 2026-08-29 · **Method:** deterministic schedule replay + invariant evaluation

> **This is a worked sample, not a client report.** The subject is this repository assessed as if it were a client system. Every finding below is reproducible by the reader with a single command, and every number is produced by the run that asserts it — see `docs/RESULTS.md`, which is generated rather than written.
>
> The point of the format is that a finding you cannot reproduce is an opinion.

Executive summary

The system's agents are well-engineered by conventional standards. Actions are authenticated with unforgeable tokens, authorized by a policy the agents do not evaluate, idempotent under retry, and individually correct given everything each agent can observe. Distributed tracing is in place. A reconciler runs.

Three of eleven tested scenarios still end with the business state wrong, and two of those three are invisible to every signal the system emits.

Severity	Finding	Evidence
Critical	No authority owns the aggregate compensation limit. Individually-valid actions combine past it.	`P2` — $1,100 against a $1,000 ceiling
Critical	Sequencing the agents does not fix it. An agent reading a lagging view is wrong even when it acts strictly after the previous one completed.	`S1` — $1,100, strictly sequential
High	Ordinary monitoring reports nothing when this happens.	`S1`, `S3` — reconciler clean, state wrong
Medium	Exposure scales with view staleness rather than being all-or-nothing. Partial catch-up is not partial safety.	`S3` — one of two events applied, still $1,100

Not found: any defect in authentication, authorization, idempotency, retry handling, or tracing. Those all work. That is what makes the findings above worth acting on rather than dismissing as immaturity.

Finding 1 — No authority for the aggregate (Critical)

What happens. Two agents each read every system available to them, each correctly concludes its action fits under the $1,000 ceiling, and both commit. The total is $1,100.

Why the existing controls do not catch it. Each control is correct at its own scope. Authorization approves *one action*. Idempotency prevents *one operation* applying twice. Neither is wrong; neither can see a sum that spans two services with no shared transaction boundary.

Reproduce:

```
make reproduce SCHEDULE=P2     # VIOLATION, overage exactly $100
```

Recommendation. Introduce a service that owns the compensation budget and must be consulted before any consequential action. Verified working in this same system:

```
make reproduce SCHEDULE=P0     # identical interleaving, CLEAN
```

Effort: the reference implementation is ~150 lines plus a table. The hard part is not the service — it is deciding *which operations count toward which limit,* which no tool can answer for you.

Finding 2 — Sequencing does not fix it (Critical)

This is the finding most likely to be missed, because the obvious remedy for Finding 1 is "stop the agents overlapping."

That remedy works for `P2`. It does not work for `S1`, where agent B begins reading *strictly after* agent A has committed and been acknowledged — and is still wrong, because B reads a projection that has not applied A's event.

Reproduce:

```
make reproduce SCHEDULE=S1     # VIOLATION despite strict sequencing
make reproduce SCHEDULE=S1C    # identical, projection caught up: CLEAN
```

The pair isolates one variable: whether the read model had caught up when the agent consulted it. Not the agent, not the ordering, not the amounts.

Recommendation. The instinct is to make the projection faster. **A faster projection is still a projection.** The fix that works is the same authority from Finding 1, because it holds authoritative state rather than a derived view:

```
make reproduce SCHEDULE=S1H    # same staleness, CLEAN
```

`S1H` asserts B's view was *exactly as stale as in S1* and the outcome was still correct.

Finding 3 — The failure is silent (High)

After `S1`: every actor succeeded, the declared interleaving executed, no work was stranded, both events were published and applied, the projection agrees with what it folded in, no duplicate keys, no orphans. The reconciler reports nothing. Every span in the trace is green.

The business state is wrong by $100.

Why the reconciler does not catch it. Deliberately, it is not told the aggregate rule. Giving a monitor the rule you are asking whether anyone knows makes the result circular. It checks what a competent team monitors: lag, drift, duplicate keys, orphans.

What this means operationally. There is no alert to tune and no dashboard to watch. A breach of this class is found by a customer, by finance, or not at all.

Recommendation. Add an invariant checker that evaluates business rules over final state at a quiescence point, separate from operational monitoring. See `docs/INVARIANT-CATALOG.md` for the form these take.

Finding 4 — Exposure tracks staleness (Medium)

Partial catch-up is not partial safety. In `S3` one of two pending events is applied before the third agent reads, and the breach still occurs.

```
make reproduce SCHEDULE=S3     # VIOLATION at partial catch-up
```

Why it matters. It removes the reassuring reading of Finding 2 — that this only bites when a view is completely cold. Any lag is exposure.

What was tested and found sound

Stated because a report that only lists problems is not an assessment.

Property	Evidence
Authentication cannot be forged or widened	tampered and wrong-key tokens rejected
Authorization is decided outside the agent	`S6` — both agents refused, nothing written
Idempotency holds under lost acknowledgement	`P3` — one effect from two create attempts
Redelivery does not inflate the projection	`S5` — at-least-once, projection unmoved
Apply order does not change the outcome	`S4` — reversed order, same result
The oracle is not trigger-happy	`P1` — sequential run reports clean

Method, and its limits

Method. Eleven named schedules, each a declared interleaving replayed deterministically. Verdicts come from an oracle that reads the databases directly under read-only credentials, evaluates only at a quiescence point, and is calibrated against a planted violation and a planted safe state before any schedule runs.

Limits, stated plainly:

- Two failure modes at one level of concurrency. Not a frequency estimate for any production system.
- Schedules are deterministic by construction. They establish that an execution *can* happen and why — not how often it does.
- The subject is a reference system, not a production one. It has three business services and no genuine crash recovery.
- Nothing here is novel. Read-then-write across uncoordinated systems predates this work by decades. What is measured is the **combination**: that these failures survive proper authentication, authorization, idempotency, strict sequencing, and competent monitoring simultaneously.

What would make this stronger for a real engagement: a naturalistic mode measuring how often the window actually opens under production-shaped load, and a second workflow to show the invariant class generalises beyond money.